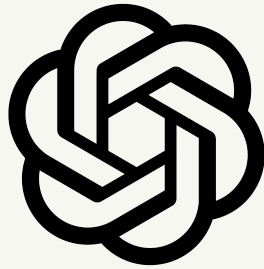


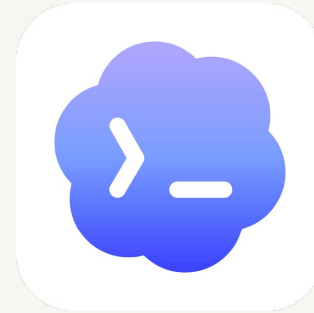
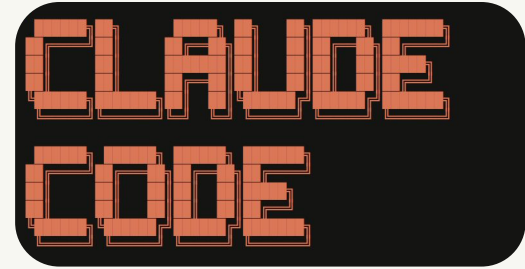
From LLM to Agent

Kashi Tuteja

2024



2026



A language model is just a function

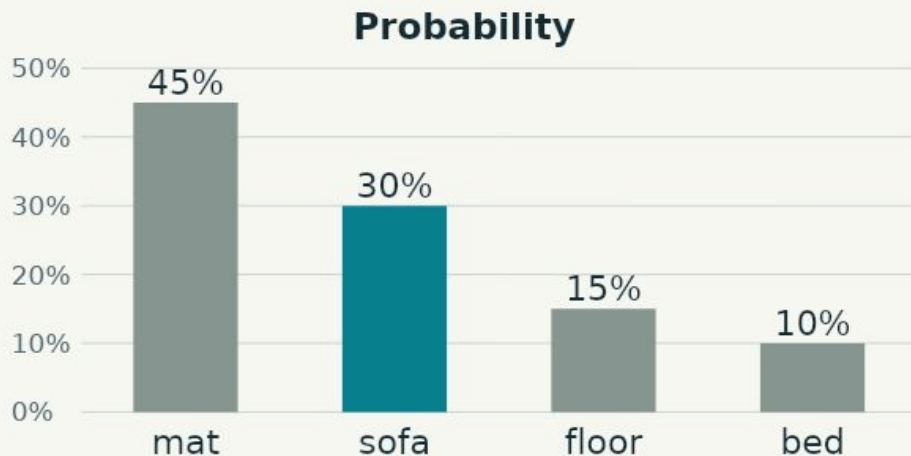
$$y = f(x)$$

`text_out = LLM(text_in)`

Sampling the next token

The cat sat on the ...

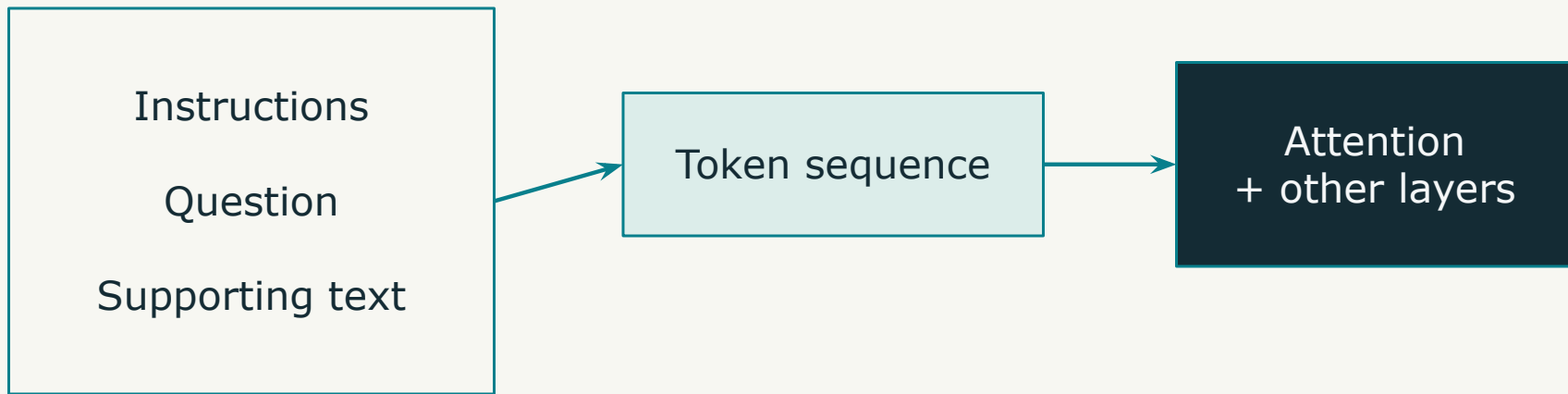
Candidate tokens	p
"mat"	45%
"sofa"	30%
"floor"	15%
"bed"	10%



`word_distribution = LLM(text_in)`

One draw: "sofa"

“Context” is an abstraction of input tokens



Roadmap

- 01 Simple LLM call
- 02 Prompts and context
- 03 Chat structure
- 04 Tool use
- 05 Agent loop
- 06 Putting it together / an example

Simple LLM call

The Atom: an API Call

```
from openai import OpenAI
```

```
o = OpenAI()
r = o.responses.create(
    model="gpt-5.5",
    instructions=system,
    input=prompt,
    max_output_tokens=300,
)
print(r.output_text)
```

```
from anthropic import Anthropic
```

```
a = Anthropic()
r = a.messages.create(
    model="claude-opus-4-8",
    system=system, max_tokens=300,
    messages=[{"role": "user",
                "content": prompt}],
)
for b in r.content:
    if b.type == "text":
        print(b.text)
```

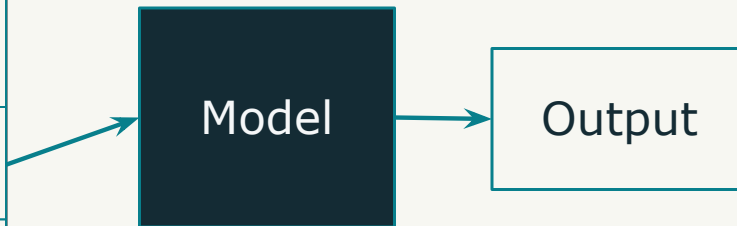
Engineering controls for different systems

System	Controls to emphasize	Purpose
Extracting invoices	Strict schema Small token budget	Predictability
Something creative	Higher temperature Room for alternatives	Unpredictability
Live assistant	Fast model Short responses	Speed
Coding	Reasoning capability Larger token budget	Complex analysis

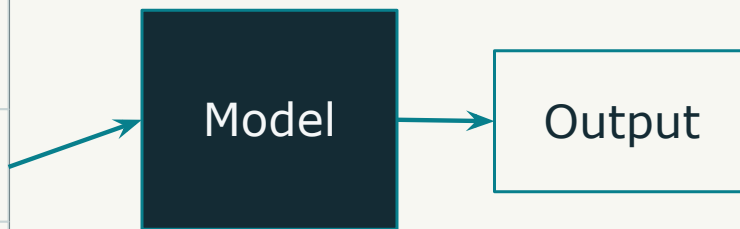
Prompts and context

The context window

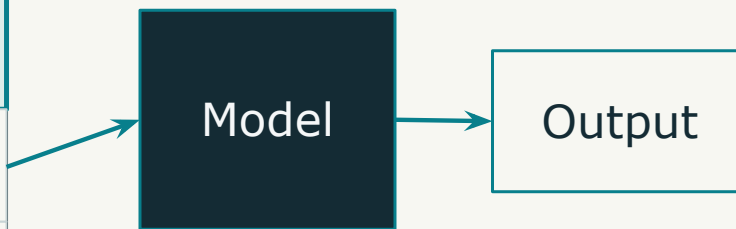
Instructions	"Use the supplied evidence."
Tool definitions	read_file(filepath: str)
Examples	ticket → BILLING_DISPUTE
Conversation + task	"Where are memories stored?"
Evidence	file contents, tool results, memory



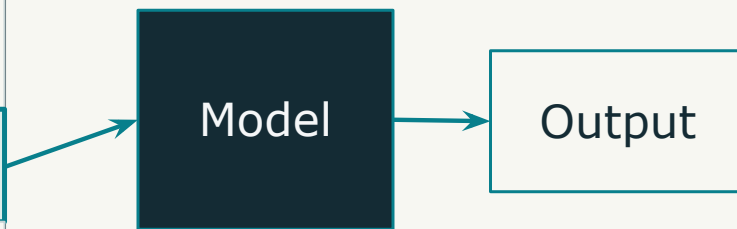
Instructions	"Use the supplied evidence."
Tool definitions	read_file(filepath: str)
Examples	ticket → BILLING_DISPUTE
Conversation + task	"Where are memories stored?"
Evidence	file contents, tool results, memory



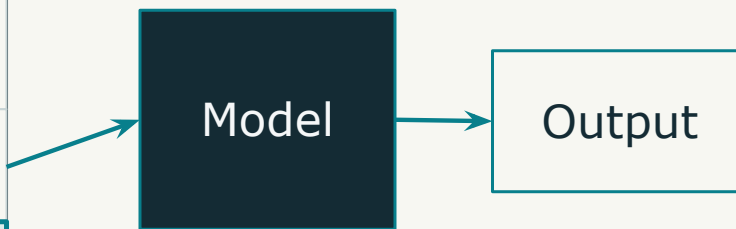
Instructions	"Use the supplied evidence."
Tool definitions	read_file(filepath: str)
Examples	ticket → BILLING_DISPUTE
Conversation + task	"Where are memories stored?"
Evidence	file contents, tool results, memory



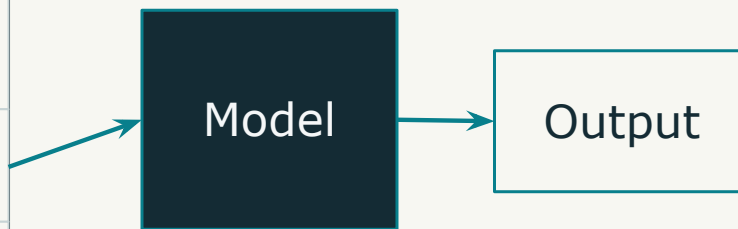
Instructions	"Use the supplied evidence."
Tool definitions	read_file(filepath: str)
Examples	ticket → BILLING_DISPUTE
Conversation + task	"Where are memories stored?"
Evidence	file contents, tool results, memory



Instructions	"Use the supplied evidence."
Tool definitions	read_file(filepath: str)
Examples	ticket → BILLING_DISPUTE
Conversation + task	"Where are memories stored?"
Evidence	file contents, tool results, memory



Instructions	"Use the supplied evidence."
Tool definitions	read_file(filepath: str)
Examples	ticket → BILLING_DISPUTE
Conversation + task	"Where are memories stored?"
Evidence	file contents, tool results, memory



"instructions": "You are an expert coding assistant operating inside Pi, an AI coding harness. You help users by reading files, executing commands, editing code, and writing new files.

Available tools:

- read: Read file contents
- bash: Execute bash commands (ls, grep, find, etc.)
- edit: Make precise file edits with exact text replacement, including multiple disjoint edits in one call
- write: Create or overwrite files
- grep: Search file contents for patterns (respects .gitignore)
- find: Find files by glob pattern (respects .gitignore)
- ls: List directory contents

In addition to the tools above, you may have access to other custom tools depending on the project.

Guidelines:

- Use read to examine files instead of cat or sed.
- Use edit for precise changes (edits[].oldText must match exactly)
- When changing multiple separate locations in one file, use one edit call with multiple entries in edits[] instead of multiple edit calls
- Each edits[].oldText is matched against the original file, not after earlier edits are applied. Do not emit overlapping or nested edits. Merge nearby changes into one edit.
- Keep edits[].oldText as small as possible while still being unique in the file. Do not pad with large unchanged regions.
- Use write only for new files or complete rewrites.
- Be concise in your responses
- Show file paths clearly when working with files

1

When asked about Pi itself, inspect the repository files directly, especially README.md, AGENTS.md, package.json, and the relevant source files under packages/.

```
<project_context>
[Project-specific instructions and guidelines]
</project_context>
```

The following skills provide specialized instructions for specific tasks. Use the read tool to load a skill's file when the task matches its description. When a skill file references a relative path, resolve it against the skill directory (parent of SKILL.md / dirname of the path) and use that absolute path in tool commands.

```
<available_skills>
  <skill>
    <name>find-skills</name>
    <description>Helps users discover and install agent skills when they ask questions like "how do I do X", "find a skill for X", "is there a skill that can...", or express interest in extending capabilities. This skill should be used when the user is looking for functionality that might exist as an installable skill.</description>
    <location>/Users/.../agents/skills/find-skills/SKILL.md</location>
  </skill>
  <skill>
    <name>runpodctl</name>
    <description>Runpod CLI to manage your GPU workloads.</description>
    <location>/Users/.../agents/skills/runpodctl/SKILL.md</location>
  </skill>
</available_skills>
```

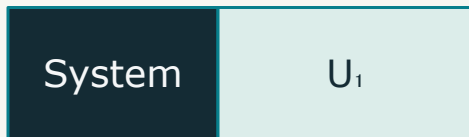
Current date: 2026-09-21
Current working directory: /Users/.../Desktop/code_repo",

2

Chat structure

Conversation

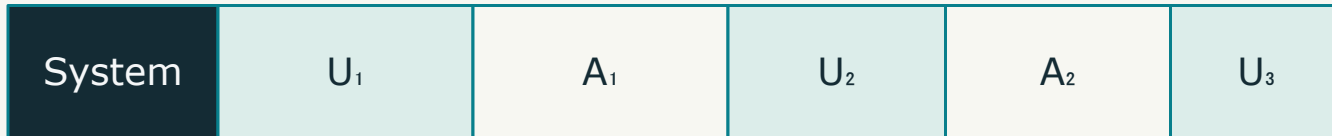
Call 1



Call 2

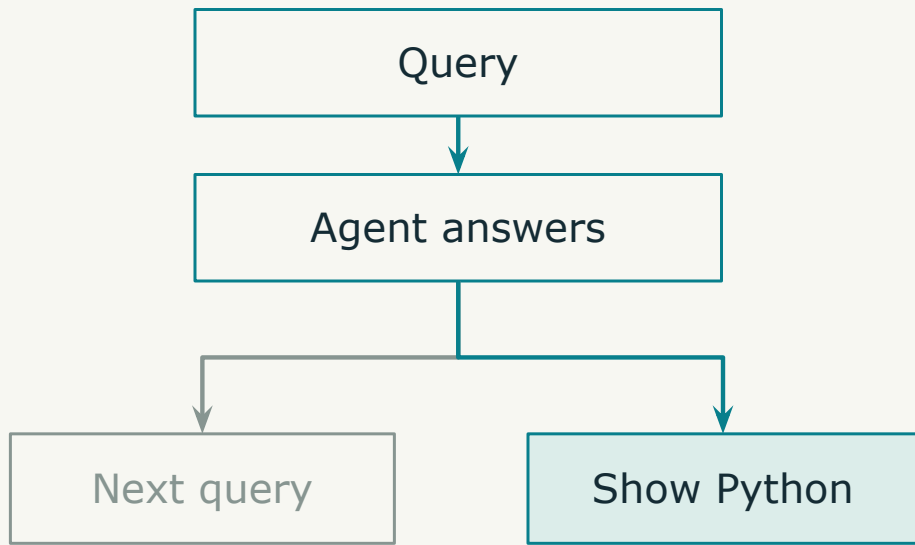


Call 3

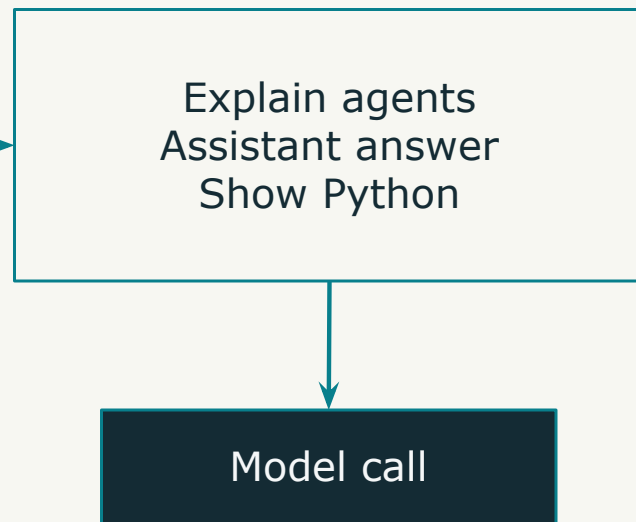


A conversation can branch

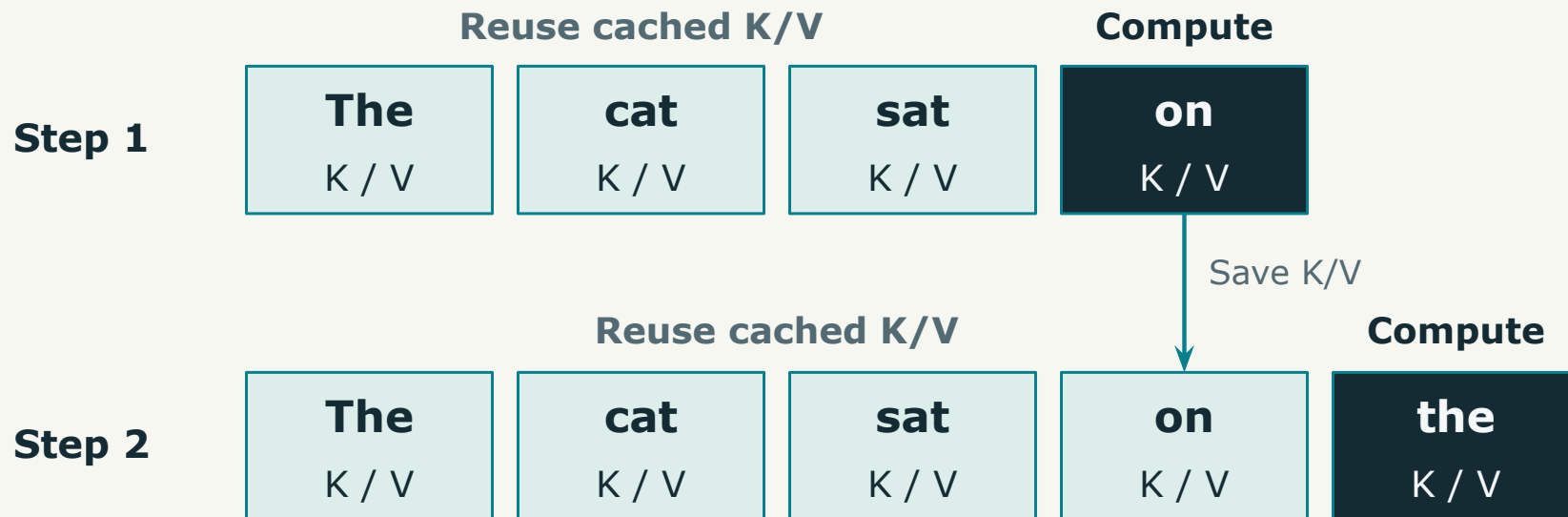
Conversation state



Selected input for the next call

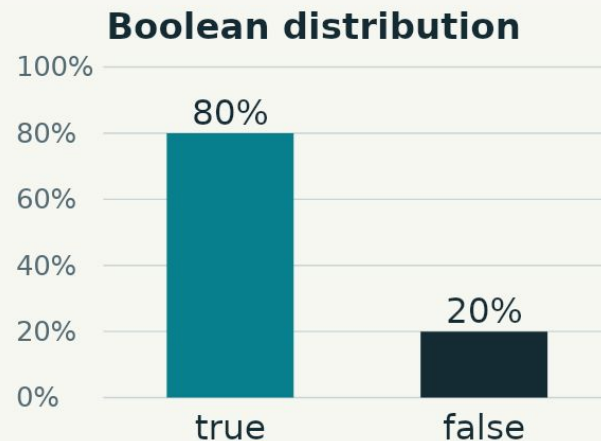
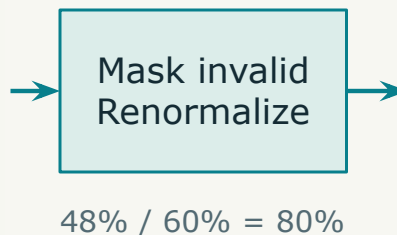
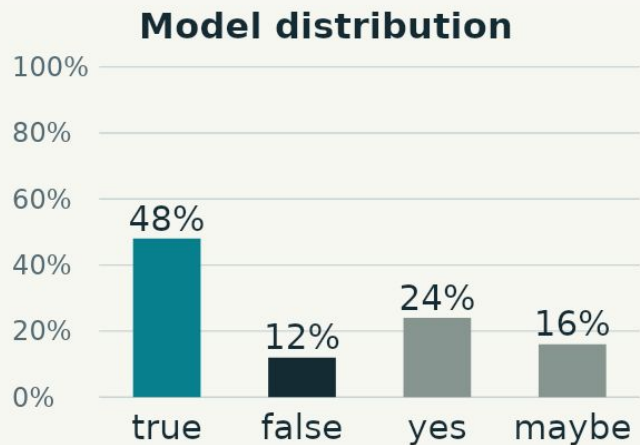


KV caching



Constrained decoding

Text input: "Available now." in_stock = ____



Outputs: {"in_stock": true}

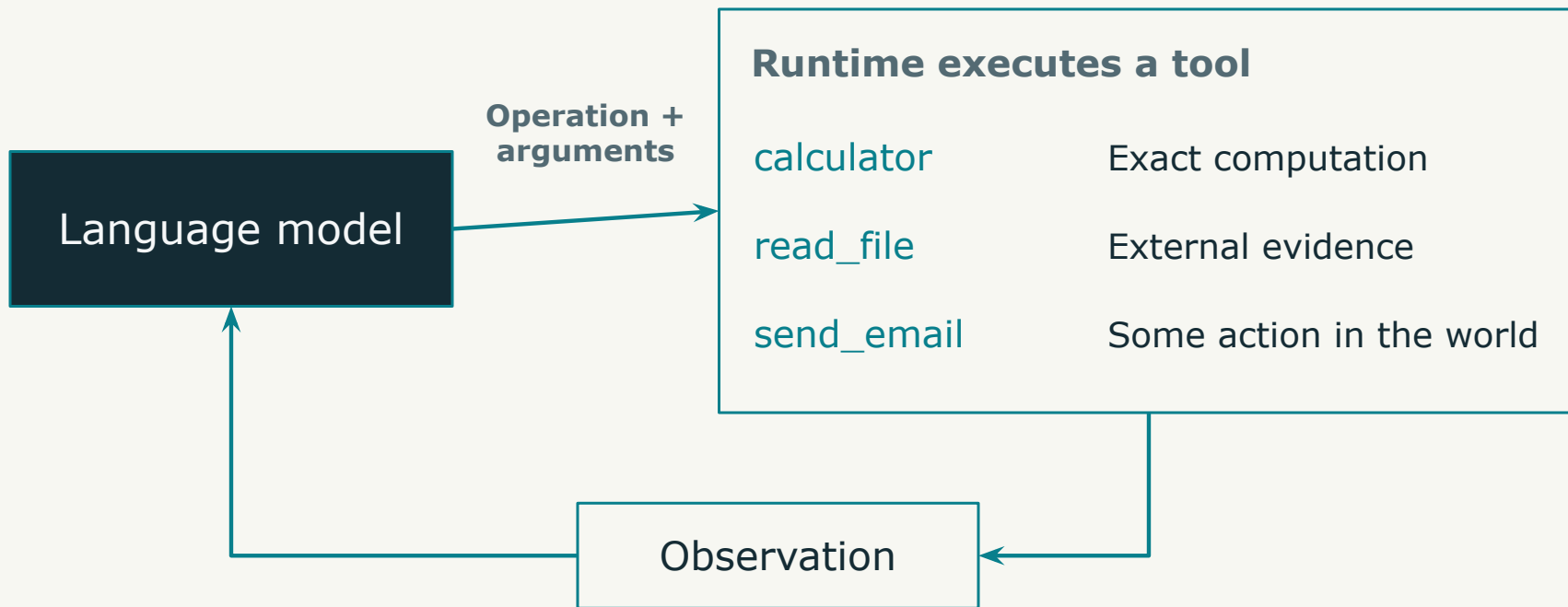
A side note



/ Jev

Tool use

The purpose of tools



Tool implementation

```
TOOLS = [{  
    "type": "function", "name": "multiply",  
    "description": "Multiply two numbers.",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "a": {"type": "number"},  
            "b": {"type": "number"},  
        },  
        "required": ["a", "b"],  
        "additionalProperties": False,  
    },  
    "strict": True,  
}]
```

EXECUTION

```
def multiply(a, b):  
    return a * b
```

The schema describes the interface, and a registry maps names to handlers.

Tool call-result function

```
def run_tools(calls):
    outputs = []
    for call in calls:
        args = json.loads(call.arguments)
        result = dispatch(call.name, args)
        outputs.append({
            "type": "function_call_output",
            "call_id": call.call_id,
            "output": json.dumps(result),
        })
    return outputs
```

call_id

Links the result to its request.

Example: database call

Context window over time

System	Instructions + tool schemas
User	"Check SKU-42. Outline delivery options."
Tool call	lookup_inventory("SKU-42") call_id="db_1"
Model	Draft delivery options while the query runs
Tool result	{"in_stock": true} call_id="db_1"
Model	Continue with the database result

Dispatch

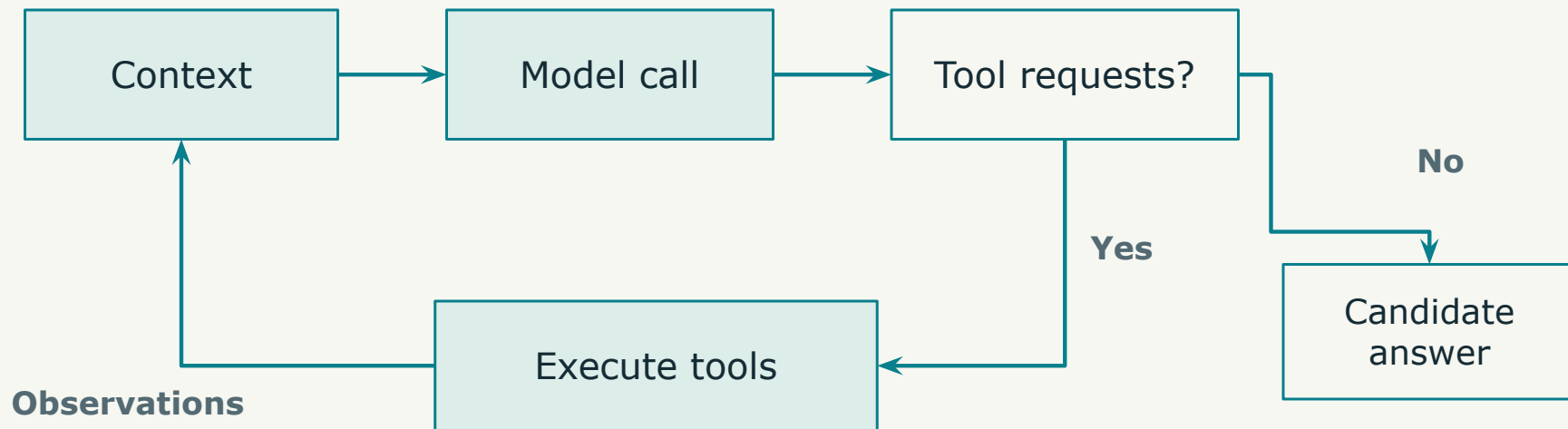
Inventory DB

```
SELECT in_stock  
FROM inventory  
WHERE sku = :sku
```

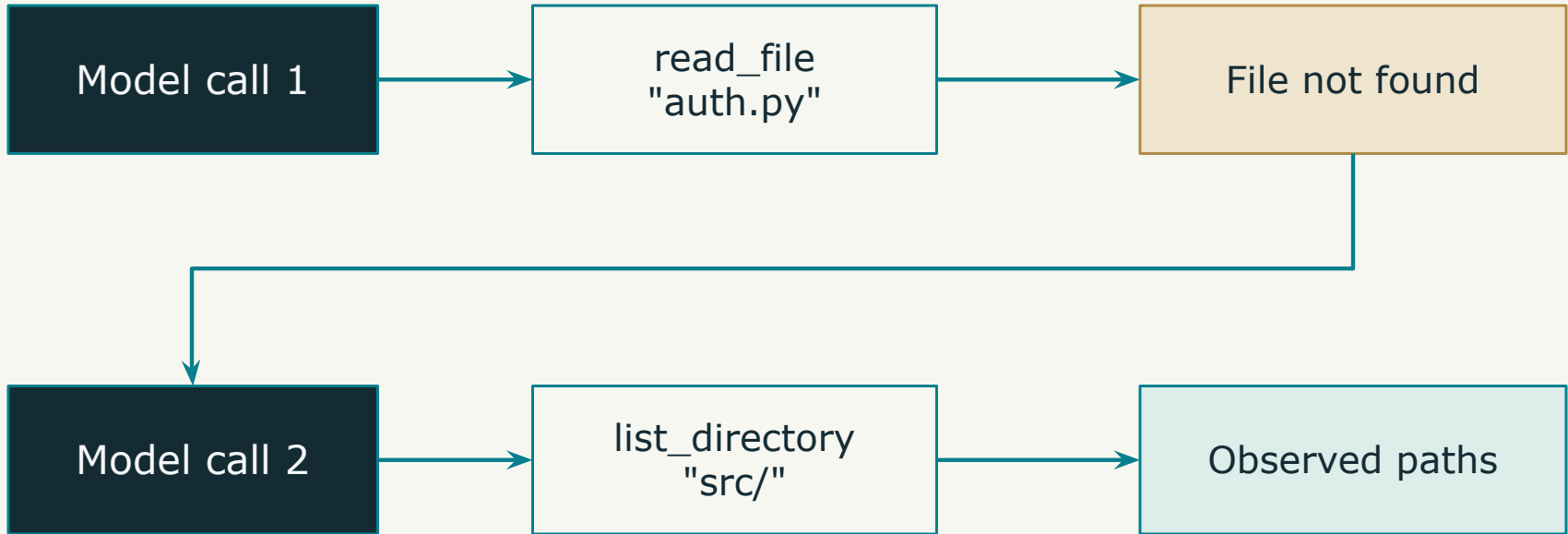
Result arrives

Agent loop

The agent loop

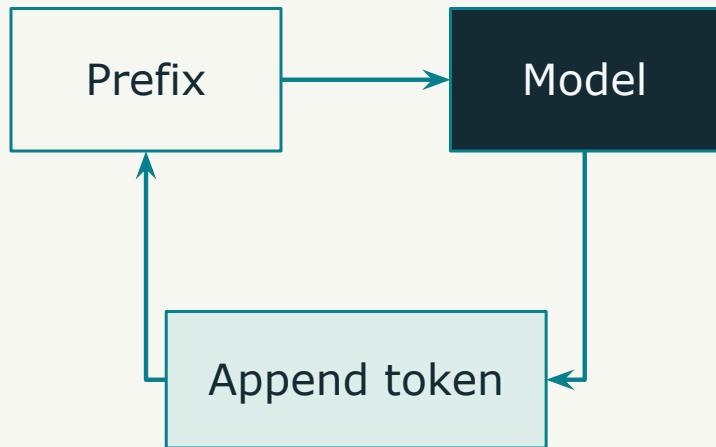


An observation changes the next action

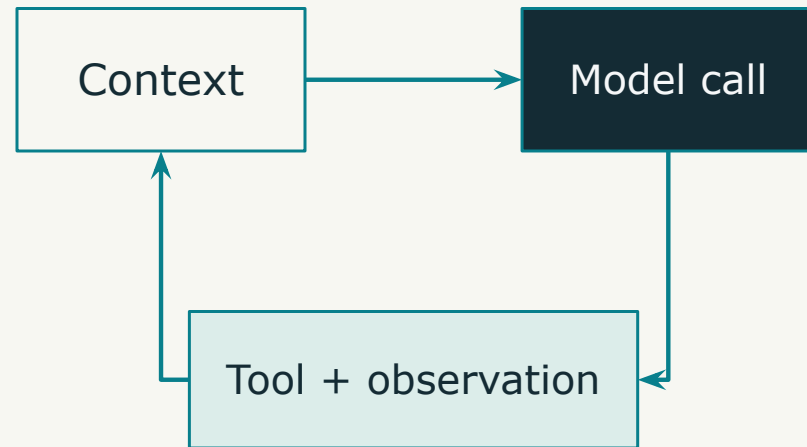


Two loops at different scales

Model loop



Agent/harness loop

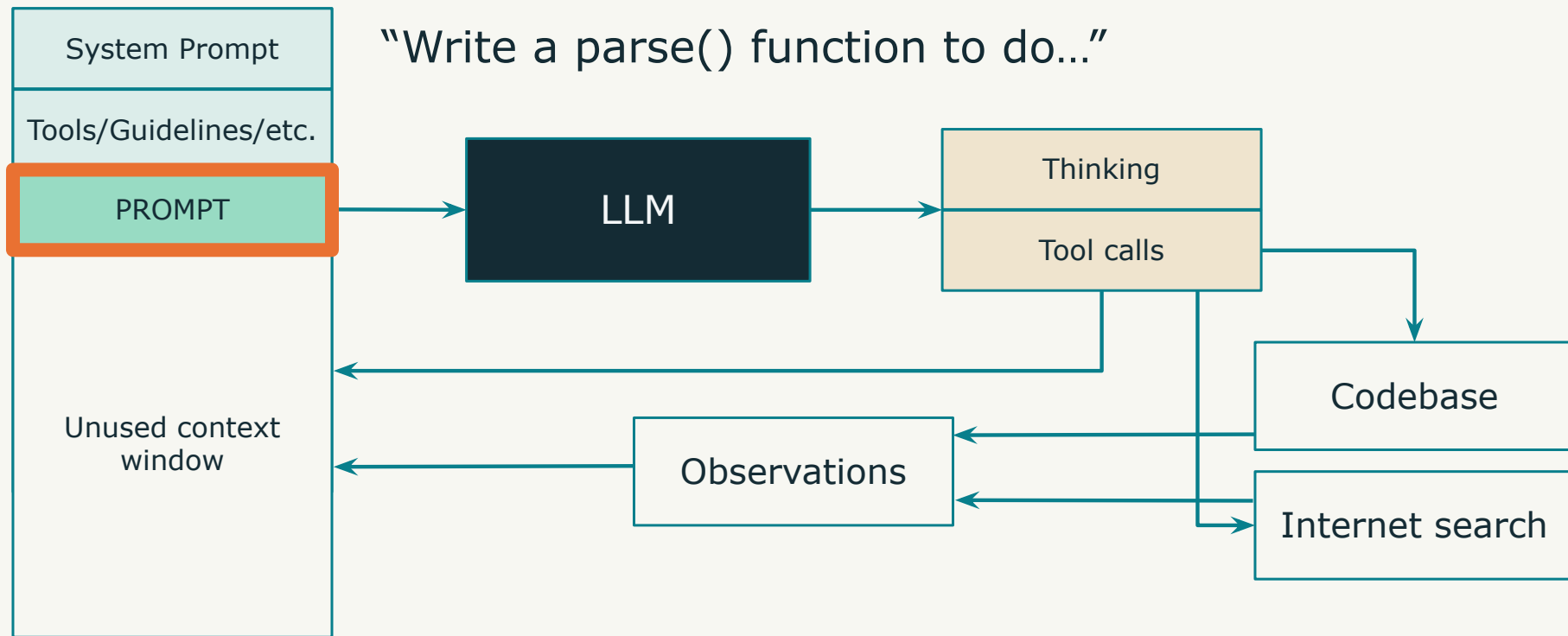
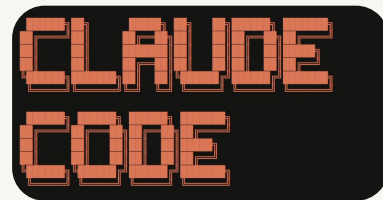


The loop in Python

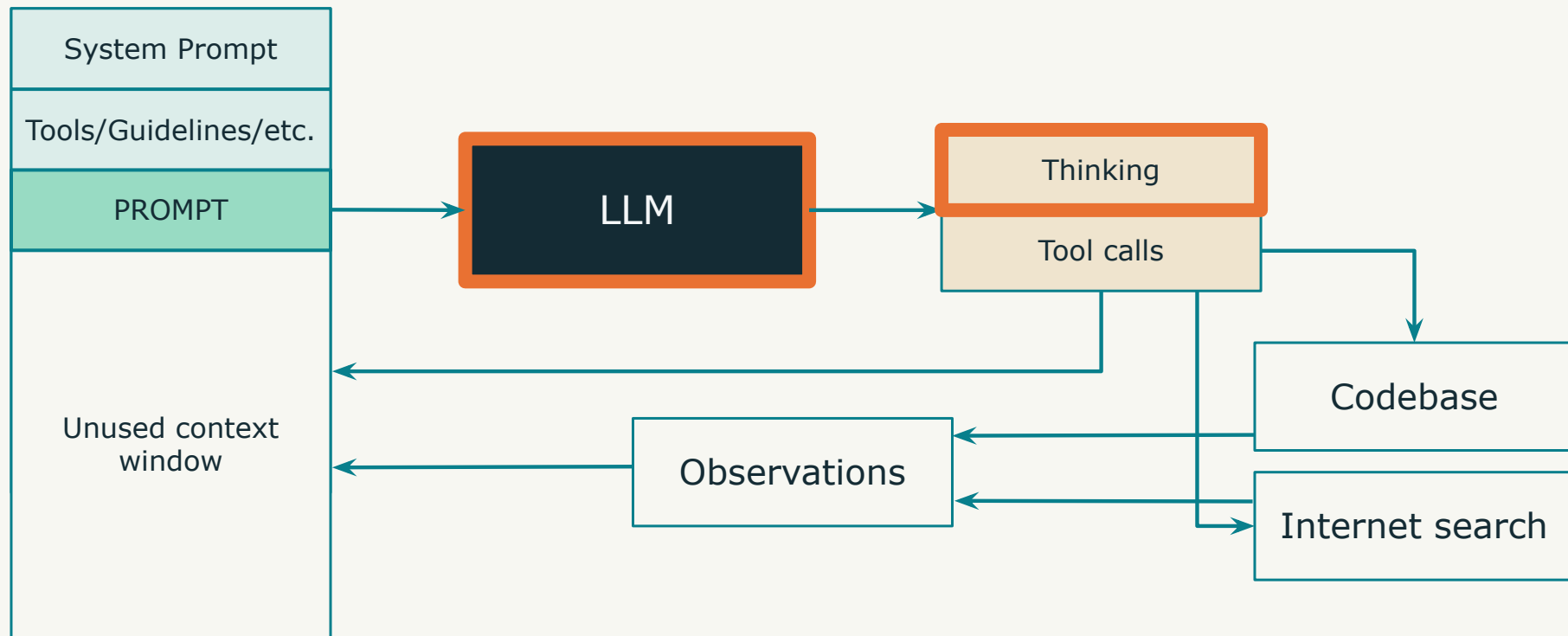
```
def agent(task):
    history = [{"role": "user", "content": task}]
    for _ in range(10):
        r = client.responses.create(
            model="gpt-5.5", instructions=SYSTEM,
            input=history, tools=TOOLS)
        if r.status != "completed":
            raise RuntimeError(r.status)
        history.extend(r.output)
        calls = [x for x in r.output
                  if x.type == "function_call"]
        if not calls:
            return r.output_text
        history.extend(run_tools(calls))
    raise RuntimeError("Step limit reached")
```

Putting it together

Example: coding agent



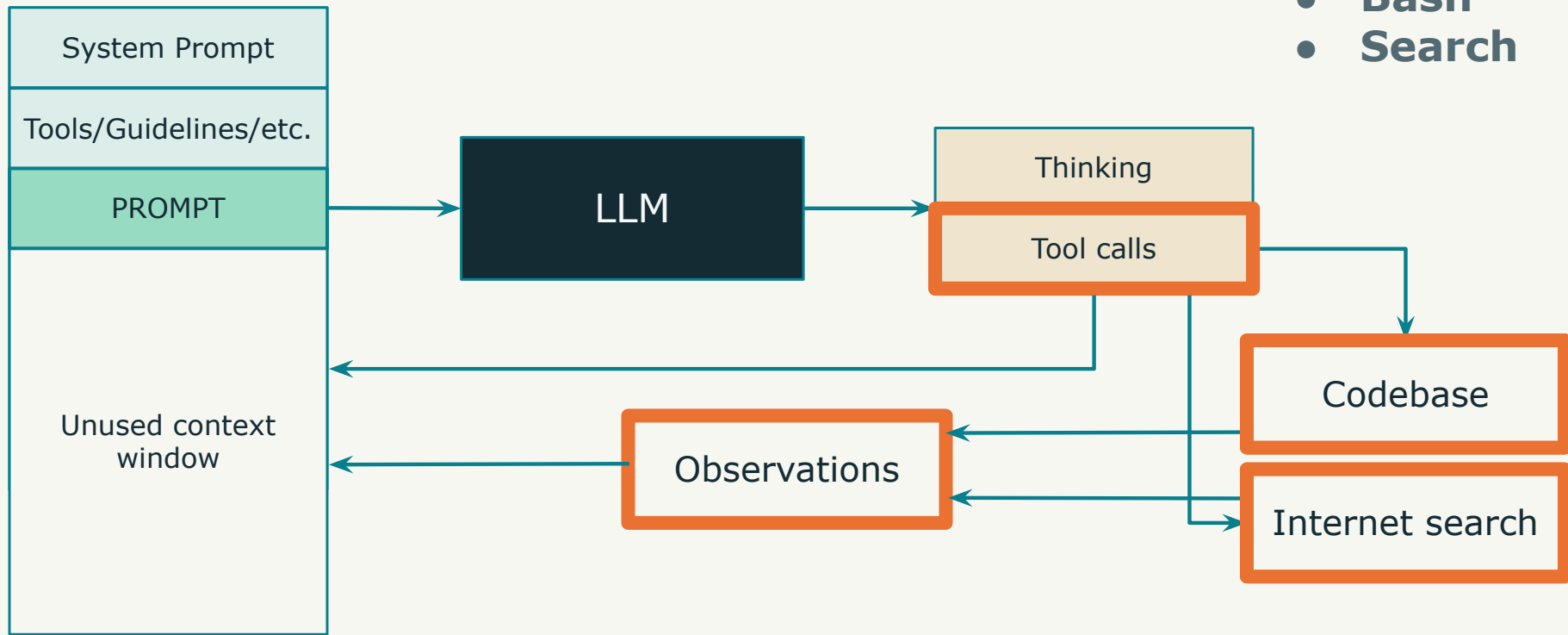
Example: coding agent



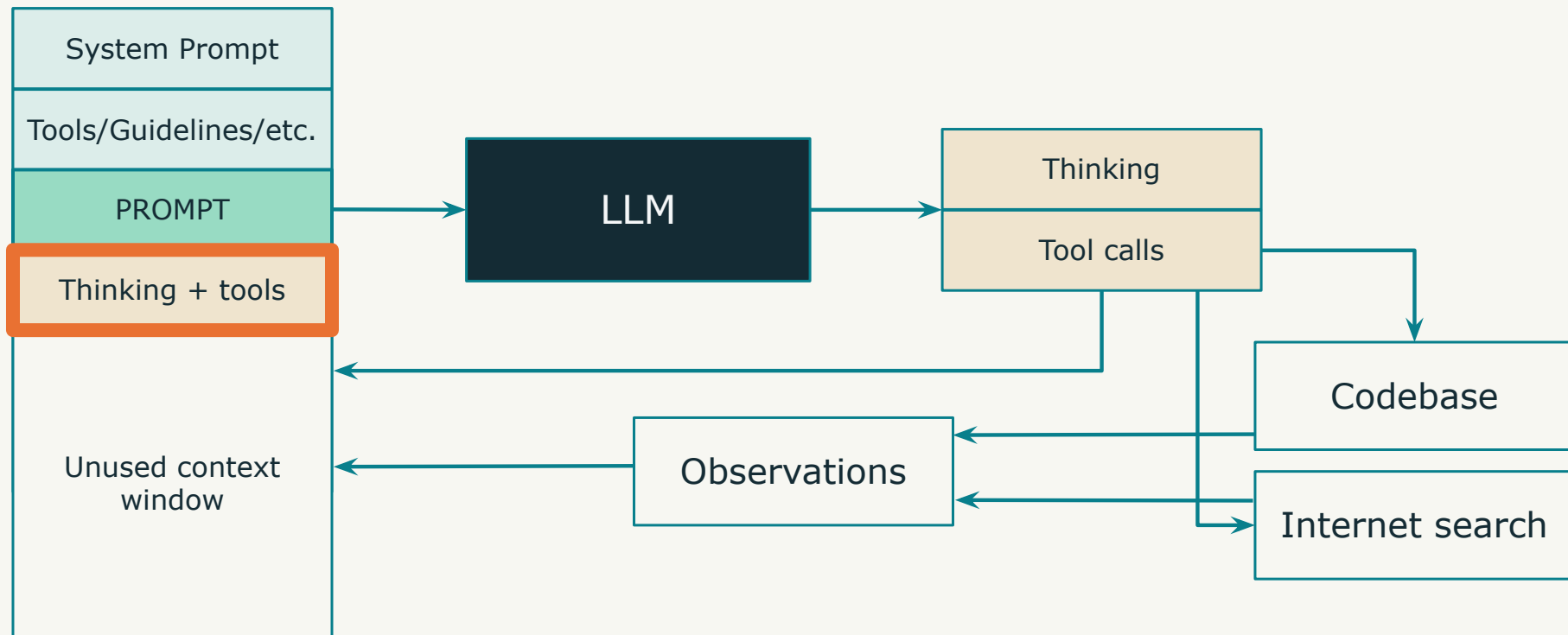
Example: coding agent

Tools:

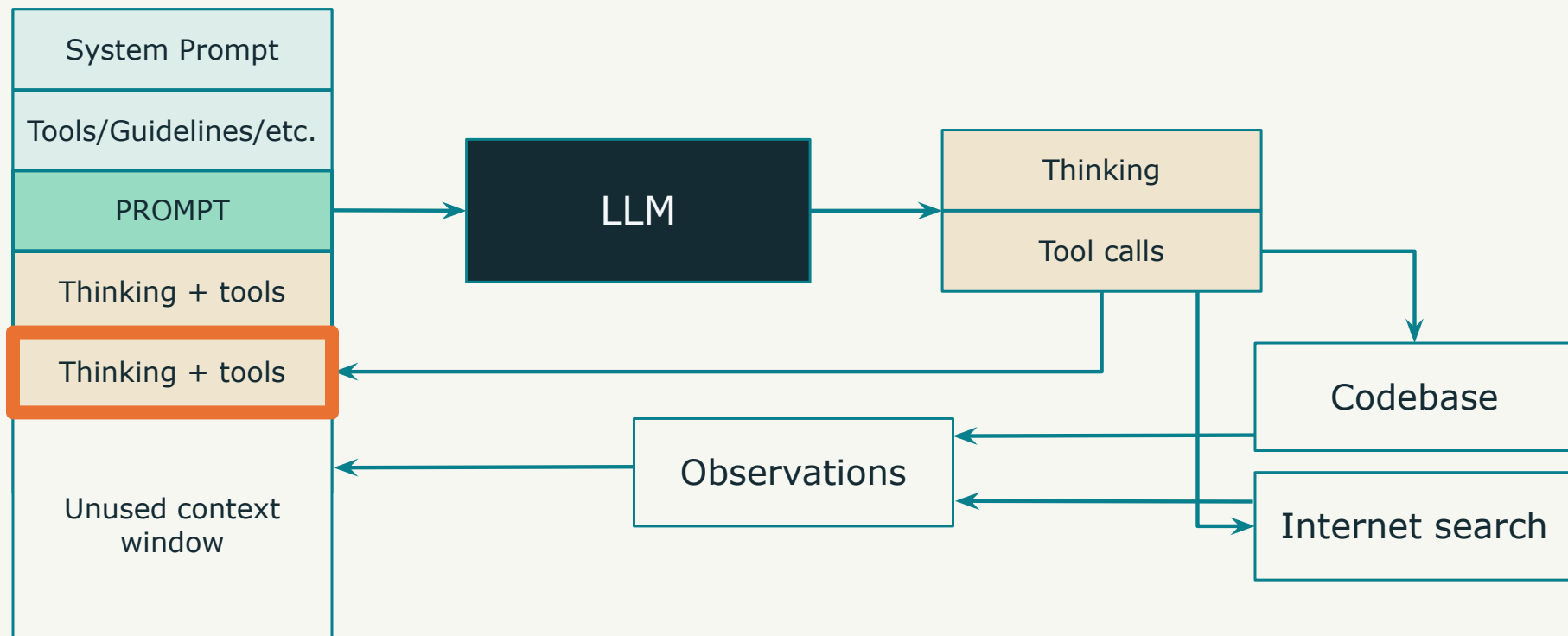
- Read
- Write
- Bash
- Search



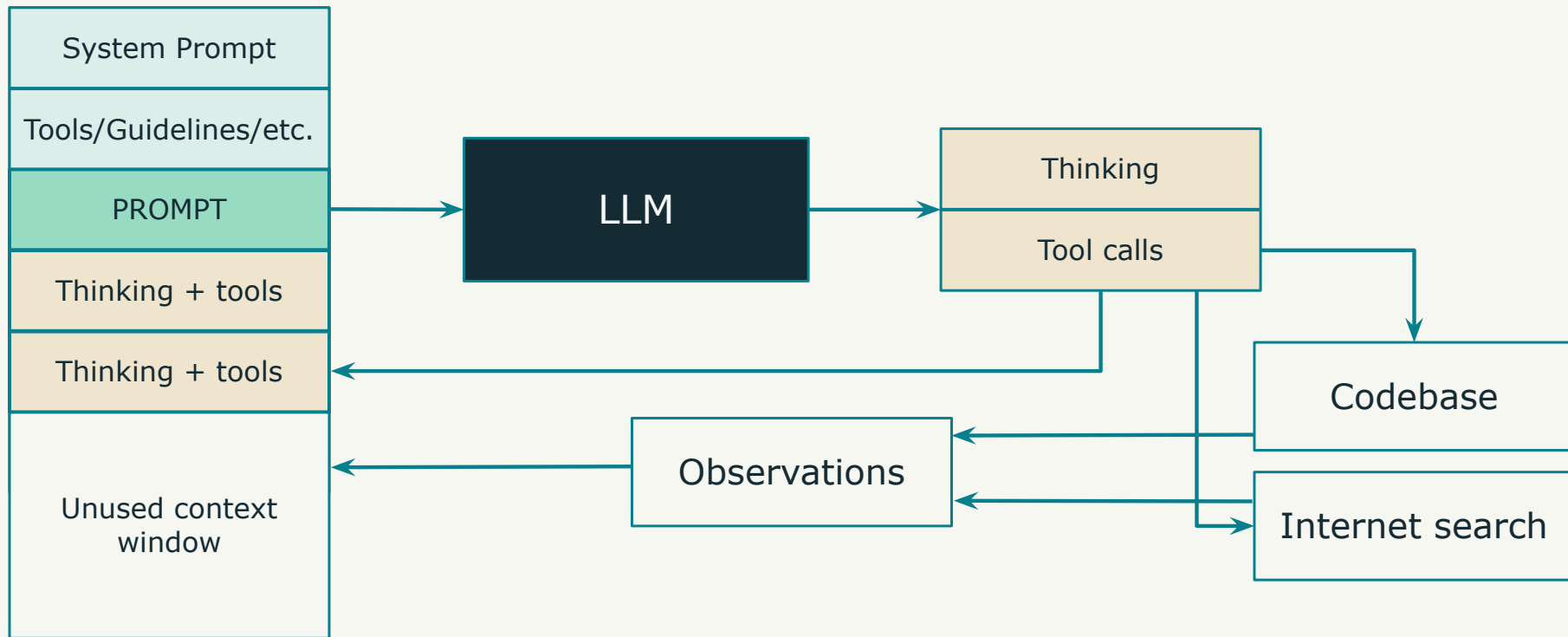
Example: coding agent



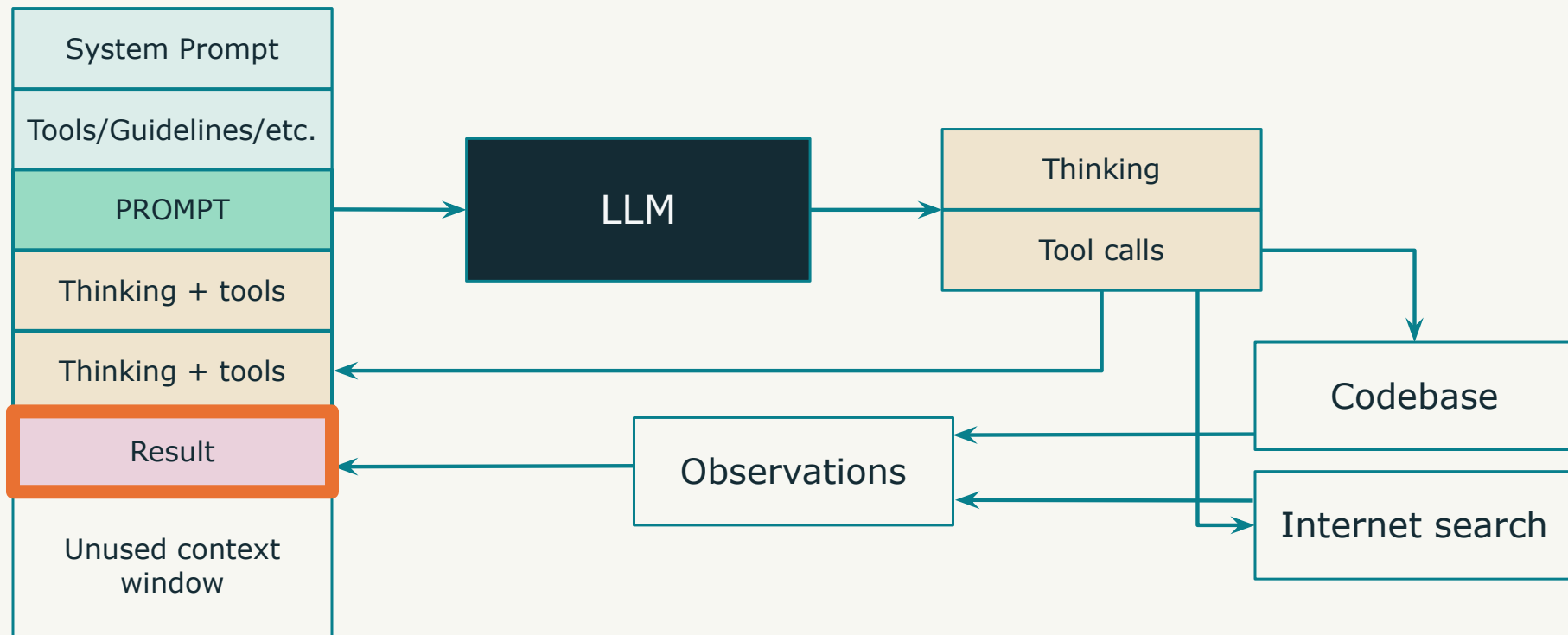
Example: coding agent



Example: coding agent



Example: coding agent

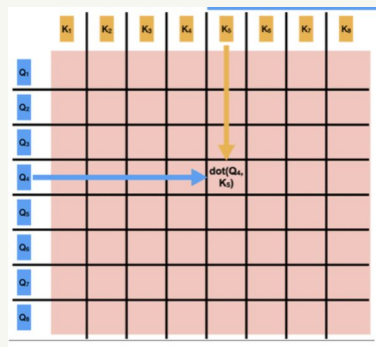


Final thoughts

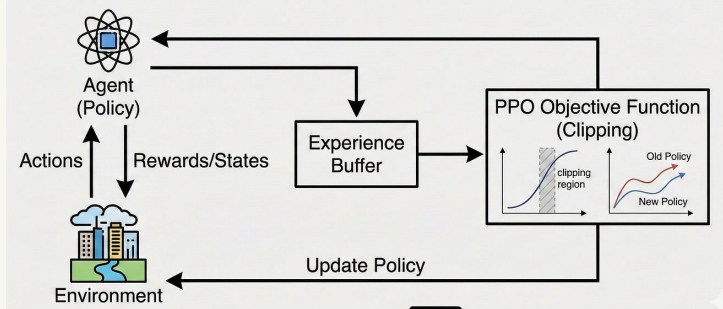
1. A model is merely a learned text-in, text-out function. Engineering the decisions, structure, and timing for those inputs and outputs is what turns it into an agent.

2. The agent loop as presented today is not the only way to do things!

Not being covered...



Proximal Policy Optimization (PPO) Algorithm



Thank you!